

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 5

з дисципліни «Технології розподілених систем та паралельних
обчислень» на тему:

**«ПАРАЛЕЛЬНІ МЕТОДИ ІНТЕГРУВАННЯ.
ВИКОРИСТАННЯ ФУНКЦІЙ КОЛЕКТИВНОГО
ОБМІНУ MPI»**

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Старший викладач

Олександр КАЧАН

1 Мета роботи

Продовжити вивчення функцій обміну бібліотеки MPI, зокрема, функцій колективного обміну. Вивчити деякі прийоми їх використання для розподілення даних та обчислень між паралельними процесорами

2 Завдання до лабораторної роботи

2.1 Паралельний метод обчислення визначених інтегралів

2.1.1 Розробити послідовні алгоритм та програму обчислення числа π з можливістю вимірювання часу роботи програми для заданого n та визначення помилки апроксимації інтеграла. Для цього використовуйте функцію вимірювання часу бібліотеки MPI, як еталонне значення числа π прийміть 3.141592653589793238462643.

Дослідіть точність та швидкодію програми обчислення π для значень кількості інтервалів $10^3, 10^4, 10^5, 10^6, 10^7, 10^9$. Результати подайте у звіті у вигляді таблиці.

2.1.2 Реалізуйте власну паралельну програму обчислення інтеграла (1) з використанням наведеної програми 1, додайте до неї функцію обчислення часу її виконання. Визначте прискорення паралельної програми для чотирьох процесорів.

2.1.3 Використовуючи опановану техніку інтегрування, розробіть послідовні алгоритм та програму обчислення інтеграла у відповідності до наведених нижче варіантів. В програмі реалізуйте введення кількості інтервалів з клавіатури. Дослідіть точність та швидкодію програми для значень кількості інтервалів $10^3, 10^4, 10^5, 10^6, 10^7, 10^9$. Результати подайте у звіті у вигляді таблиці.

Обчислити інтеграл $\int_0^\pi \cos^2(x) dx$

2.2 Програма для обчислення числа π методом Монте-Карло

2.2.1 Програму 5.2 доповніть функцією обчислення часу її виконання та функцією обчислення дисперсії σ^2 випадкової величини значення π

$$\sigma^2 \approx \frac{1}{N+1} \sum_{i=1}^N (\xi_i - \xi_n)^2$$

причому, оскільки середнє значення ξ_n заздалегідь не відомо, зручніше використати формулу

$$\sigma^2 \approx \frac{1}{N+1} \left[\sum_{i=1}^N \xi_i^2 - \frac{1}{N} \left(\sum_{i=1}^N \xi_i \right)^2 \right]$$

2.2.2 Використовуючи формули визначити N таке, щоб похибка ε обчислення числа π наближено дорівнювала 10^{-4}

2.2.3 Для випадку використання двох та чотирьох процесів, за допомогою доповненої програми обчислити число π , показати, що виконується відношення 1.

2.2.4 Визначити прискорення паралельної програми для двох та чотирьох процесорів.

2.2.5 Проведені розрахунки та експериментальні результати навести у звіті.

2.3 Дано послідовний алгоритм обчислення інтеграла методом Монте-Карло.

2.3.1 Розробити паралельну програму інтегрування методом Монте-Карло, використовуючи програму 5.3.

2.3.2 Програму 5.3 доповніть функцією обчислення часу її виконання та функцією обчислення дисперсії σ^2 випадкової величини значення π

$$\sigma^2 \approx \frac{1}{N+1} \sum_{i=1}^N (\xi_i - \xi_n)^2$$

причому, оскільки середнє значення ξ_n заздалегідь не відомо, зручніше використати формулу

$$\sigma^2 \approx \frac{1}{N+1} \left[\sum_{i=1}^N \xi_i^2 - \frac{1}{N} \left(\sum_{i=1}^N \xi_i \right)^2 \right]$$

2.3.3 Використовуючи формули визначити N таке, щоб похибка ε обчислення числа π наближено дорівнювала 10^{-4}

2.3.4 Підібрати для знайденого N параметри N_in_proc та n_in_proc для випадку використання двох та чотирьох процесів і за допомогою доповненої програми обчислити число π , показати, що виконується відношення 1.

2.3.5 Визначити прискорення паралельної програми для двох та чотирьох процесорів.

2.3.6 Проведені розрахунки та експериментальні результати навести у звіті.

2.4 Розробити паралельну програму для обчислення кратного інтеграла $\iint_S f(x,y)dx dy$, де S – чверть кола $x^2 + y^2 \leq R^2, x \geq 0; y \geq 0$; Точне значення інтеграла дорівнює $\frac{1}{8}R^4$

3 Виконання лабораторної роботи

Для виконання завдання 2.1.1 було взято код програми один та переписано його в послідовний варіант. В коді було змінено логіку отримання параметра N , тепер він буде передаватись через параметри. Також було додано логіку вимірювання часу. Оскільки логіка послідовна, то кількість процесів не має значення. На рисунку 1 показано результати запуску алгоритму.

```
C:\home\university-4\distributed-systems\lb5\x64\Debug>mpirun -n 8 lb5.exe 10000000
4: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.142833sec
3: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.158158sec
2: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.144039sec
7: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.132294sec
0: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.158827sec
6: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.144250sec
5: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.168034sec
1: pi is approximately 3.1415926535897309;      Error is 0.00000000000000622      Time: 0.148762sec
```

Рисунок 1 – Запуск послідовного алгоритму

В таблиці 1 показані результати проведених тестів програми та їх результатів.

Таблиця 1 – Результати тестування послідовного алгоритму

Кількість інтервалів (n)	Обчислене значення π	Помилка апроксимації	Час виконання (сек)
10^3	3,141593	0,0000000833333296	0,000012
10^4	3,141593	0,0000000008333410	0,000112
10^5	3,141593	0,0000000000083684	0,001122
10^6	3,141593	0,0000000000000289	0,010892
10^7	3,141593	0,0000000000000622	0,111196
10^9	3,141593	0,0000000000001776	11,231025

Як можна побачити з таблиці, при збільшені кількості елементів час значно зростає, а також значно збільшується точність вимірів до 10^7 степені N, оскільки далі вже дає про себе знати накопичення помилки з роботою з типом double.

Для виконання завдання 2.1.2 було взято вихідний код з методичних вказівок, виконано його рефакторинг і додано в нього логіку визначення часу.

На рисунку 2 показано запуск паралельних обчислень

```
C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 4 lb5.exe 1000000000
0: pi is approximately 3.1415926535897682;      p = 4   Error is  0.0000000000000249   Time: 2.832542sec
```

Рисунок 2 – Приклад виконання паралельних обчислень числа π

В таблиці 2 наведені результати вимірів швидкості паралельних обчислень. Обчислення відбувались з $N = 10^9$.

Таблиця 2 – Заміри паралельних обчислень

P	π	Error	T1	TN	S
1	3,141593	0,00000000000001776	10,9246	10,9246	1,00
2	3,141593	0,00000000000002141	10,9246	5,5006	1,99
3	3,141593	0,00000000000001217	10,9246	3,6806	2,97
4	3,141593	0,0000000000000249	10,9246	2,7796	3,93

На рисунку 3 показаний графік залежності прискорення від кількості процесорів

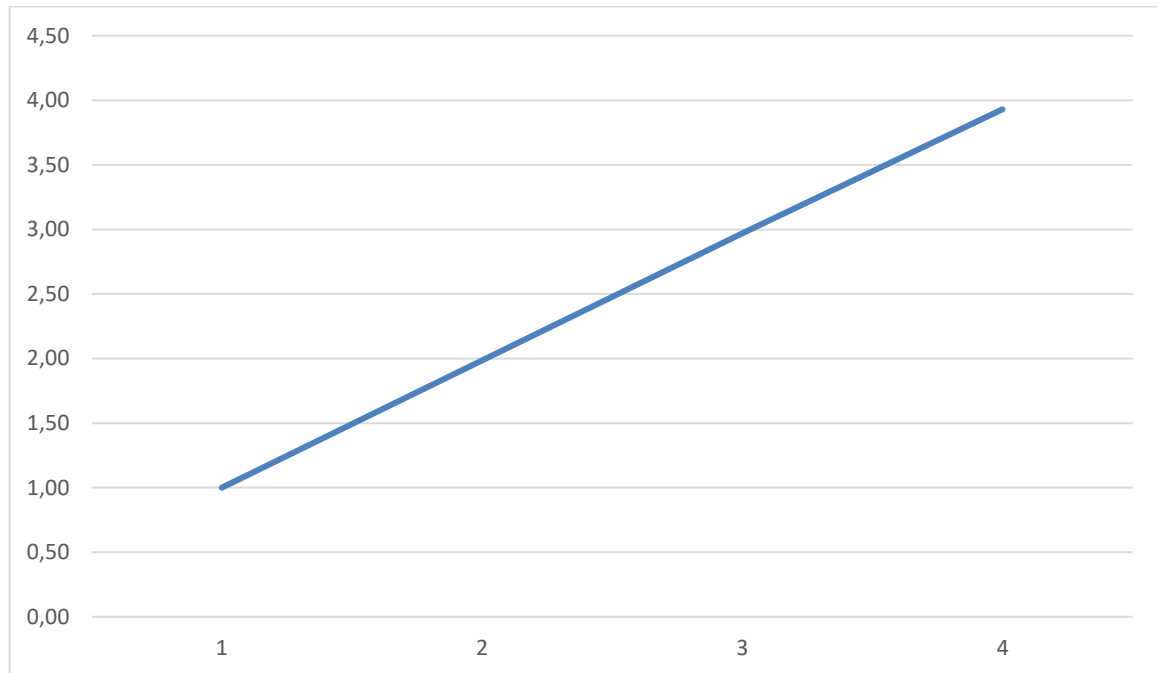


Рисунок 3 – Залежність прискорення від кількості процесорів

Для виконання наступного завдання 2.1.3 було взято за основу послідовний алгоритм з завдання 2.1.1 та адаптовано до нових умов.

На рисунку 4 показаний приклад запуску алгоритму.

```
C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 1 lb5.exe 2
0: Answer is approximately 1.5707963267948966; Error is 0.0000000000000000 Time: 0.000006sec N = 2

C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 1 lb5.exe 10
0: Answer is approximately 1.5707963267948966; Error is 0.0000000000000000 Time: 0.000006sec N = 10

C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 1 lb5.exe 1000
0: Answer is approximately 1.5707963267948957; Error is 0.0000000000000009 Time: 0.000028sec N = 1000

C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 1 lb5.exe 1000000
0: Answer is approximately 1.5707963267949689; Error is 0.0000000000000724 Time: 0.022951sec N = 1000000
```

Рисунок 4 – Запуск алгоритму на знаходження косинусу

Цікаво, що $N = 2$ точність результату вже є дуже і дуже гарною, але при збільшені N точність падає через помилку, яка накопичується в системі.

В таблиці 3 показані заміри функції згідно з завданням

Таблиця 3 – Заміри розрахунку інтегралу косинусів

N	Value	Error	Time
10 ³	1,570796327	0,00000000000000009	0,000028
10 ⁴	1,570796327	0,00000000000000036	0,000233
10 ⁵	1,570796327	0,00000000000000016	0,002299
10 ⁶	1,570796327	0,00000000000000724	0,022984
10 ⁷	1,570796327	0,00000000000001288	0,23162
10 ⁹	1,570796327	0,00000000000001779	23,245865

Як можна побачити з таблиці при збільшені N точність розрахунків тільки зменшується.

В завданні 2.2 було взято код програми 2 та виправлено в ньому помилки. На рисунку 5 показаний запуск другої програми

```
C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 1 lb5.exe 10
pi=3.1428120000000002; error=0.0012193464102070, points=1000000
pi=3.1431600000000000; error=0.0015673464102068, points=2000000
pi=3.1419120000000000; error=0.0003193464102069, points=3000000
pi=3.1421690000000000; error=0.0005763464102069, points=4000000
pi=3.1419920000000001; error=0.0003993464102070, points=5000000
pi=3.1416746666666668; error=0.0000820130768737, points=6000000
pi=3.1418074285714286; error=0.0002147749816355, points=7000000
pi=3.1417185000000001; error=0.0001258464102070, points=8000000
pi=3.1415053333333334; error=0.0000873202564597, points=9000000
pi=3.1417031999999998; error=0.0001105464102067, points=10000000
```

Рисунок 5 – Запуск другого алгоритму

Як можна побачити, при кожній ітерації значення PI все більше уточняється, але оскільки метод є рандомним, то іноді значення стає менше

Для виконання завдання 2.2.1 в код було додано розрахунок дисперсії згідно формул. Результат виконання показаний на рисунку 6.

```

C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 1 lb5.exe 10
pi=3.1420880000000002; error=0.0004953464102071, points=1000000
Step 2: pi=3.1421920000, sigma^2=5.4079976053e-09
pi=3.1421920000000001; error=0.0005993464102070, points=2000000
Step 3: pi=3.1416933333, sigma^2=6.9207700903e-08
pi=3.1416933333333334; error=0.0001006797435403, points=3000000
Step 4: pi=3.1417660000, sigma^2=5.8807219242e-08
pi=3.1417660000000001; error=0.0001733464102069, points=4000000
Step 5: pi=3.1412704000, sigma^2=1.3239974450e-07
pi=3.1412703999999998; error=0.0003222535897933, points=5000000
Step 6: pi=3.1407780000, sigma^2=2.8066425841e-07
pi=3.1407780000000001; error=0.0008146535897930, points=6000000
Step 7: pi=3.1410120000, sigma^2=2.8867527391e-07
pi=3.1410119999999999; error=0.0005806535897932, points=7000000
Step 8: pi=3.1408190000, sigma^2=3.1292519915e-07
pi=3.1408190000000000; error=0.0007736535897931, points=8000000
Step 9: pi=3.1409782222, sigma^2=2.9878613184e-07
pi=3.1409782222222220; error=0.0006144313675711, points=9000000
Step 10: pi=3.1408612000, sigma^2=2.9458177330e-07
pi=3.1408611999999998; error=0.0007314535897933, points=10000000

```

Рисунок 6 – Дисперсії

Як можна побачити з рисунку 6 дисперсії мають дуже маленьке значення, а отже результат є достатньо стабільним

Для виконання завдання 2.2.2 потрібно розрахувати формулу.

$$\varepsilon \approx 3 \frac{\sigma}{\sqrt{N}} = 3 \frac{4 \sqrt{\frac{\pi}{4} \left(1 - \frac{\pi}{4}\right)}}{\sqrt{N}}; 10^{-4} = \frac{1.64}{\sqrt{N}} \Rightarrow \sqrt{N} = 16400;$$

$$N = 16400^2 = 268\,960\,000$$

Для завдання 2.2.3 було обрано, що n_{in_proc} , тобто точок за крок буде 10 мільйонів. Для двох процесорів було обрано кількість кроків = 120, а для 4-х 60.

Результати запуску алгоритму для одного процесору та $N = 240$ показано на рисунку 7, для двох процесорів та $N = 120$ показано на рисунку 8, а для чотирьох процесорів та $N = 60$ на рисунку 9.


```

Step 195: pi=3.14138974426 sigma^2=7.58421888051e-09 points=1950000000
Step 196: pi=3.14137977114 sigma^2=7.4703562766e-09 points=1960000000
Step 197: pi=3.1413826761 sigma^2=7.4399636384e-09 points=1970000000
Step 198: pi=3.1413839273 sigma^2=7.4183019851e-09 points=1980000000
Step 199: pi=3.1413864724 sigma^2=7.3819103019e-09 points=1990000000
Step 200: pi=3.1413928160 sigma^2=7.3562056681e-09 points=2000000000
Step 201: pi=3.1413930149 sigma^2=7.3311252891e-09 points=2010000000
Step 202: pi=3.1413974653 sigma^2=7.3083985210e-09 points=2020000000
Step 203: pi=3.1413972867 sigma^2=7.2856716479e-09 points=2030000000
Step 204: pi=3.1413978078 sigma^2=7.2633042131e-09 points=2040000000
Step 205: pi=3.1414000390 sigma^2=7.2421892796e-09 points=2050000000
Step 206: pi=3.1413986175 sigma^2=7.2204019067e-09 points=2060000000
Step 207: pi=3.1413987710 sigma^2=7.1987741852e-09 points=2070000000
Step 208: pi=3.1413980115 sigma^2=7.1768424633e-09 points=2080000000
Step 209: pi=3.1413966813 sigma^2=7.1543542372e-09 points=2090000000
Step 210: pi=3.1413970838 sigma^2=7.1321573634e-09 points=2100000000
Step 211: pi=3.1413951014 sigma^2=7.1091411276e-09 points=2110000000
Step 212: pi=3.1413958868 sigma^2=7.0865973690e-09 points=2120000000
Step 213: pi=3.1413967117 sigma^2=7.0645387066e-09 points=2130000000
Step 214: pi=3.1413965738 sigma^2=7.0425185058e-09 points=2140000000
Step 215: pi=3.1414028353 sigma^2=7.0232094616e-09 points=2150000000
Step 216: pi=3.1414011500 sigma^2=7.0035195330e-09 points=2160000000
Step 217: pi=3.1414010304 sigma^2=6.9830329687e-09 points=2170000000
Step 218: pi=3.1413940404 sigma^2=6.9610754600e-09 points=2180000000
Step 219: pi=3.1413951489 sigma^2=6.9389084733e-09 points=2190000000
Step 220: pi=3.1413907800 sigma^2=6.9151124765e-09 points=2200000000
Step 221: pi=3.1413879240 sigma^2=6.8904248771e-09 points=2210000000
Step 222: pi=3.1413895676 sigma^2=6.8664874611e-09 points=2220000000
Step 223: pi=3.1413867534 sigma^2=6.8417230560e-09 points=2230000000
Step 224: pi=3.1413822643 sigma^2=6.8157410597e-09 points=2240000000
Step 225: pi=3.1413810471 sigma^2=6.7896012622e-09 points=2250000000
Step 226: pi=3.1413825681 sigma^2=6.76400879327e-09 points=2260000000
Step 227: pi=3.1413814379 sigma^2=6.7384381968e-09 points=2270000000
Step 228: pi=3.1413833754 sigma^2=6.7135252893e-09 points=2280000000
Step 229: pi=3.1413847755 sigma^2=6.6891998996e-09 points=2290000000
Step 230: pi=3.1413856289 sigma^2=6.6652934816e-09 points=2300000000
Step 231: pi=3.1413844294 sigma^2=6.6411995130e-09 points=2310000000
Step 232: pi=3.1413810552 sigma^2=6.6163495346e-09 points=2320000000
Step 233: pi=3.1413813150 sigma^2=6.5917431819e-09 points=2330000000
Step 234: pi=3.1413826923 sigma^2=6.5676778808e-09 points=2340000000
Step 235: pi=3.1413839540 sigma^2=6.5441272610e-09 points=2350000000
Step 236: pi=3.1413820153 sigma^2=6.5202197640e-09 points=2360000000
Step 237: pi=3.1413836928 sigma^2=6.4969214490e-09 points=2370000000
Step 238: pi=3.1413872000 sigma^2=6.4747791264e-09 points=2380000000
Step 239: pi=3.1413843130 sigma^2=6.4519630572e-09 points=2390000000
Step 240: pi=3.1413829900 sigma^2=6.4289516676e-09 points=2400000000
Time = 233.024007

```

Рисунок 7 – Запуск для одного процесору з N = 240

```

Step 95: pi=3.1413977242 sigma^2=3.7029532537e-09 points=1900000000
Step 96: pi=3.1413979000 sigma^2=3.6711741670e-09 points=1920000000
Step 97: pi=3.1413969320 sigma^2=3.6404384692e-09 points=1940000000
Step 98: pi=3.1413978510 sigma^2=3.6096918810e-09 points=1960000000
Step 99: pi=3.1413970283 sigma^2=3.5798674320e-09 points=1980000000
Step 100: pi=3.1413905620 sigma^2=3.5542984044e-09 points=2000000000
Step 101: pi=3.1413877386 sigma^2=3.5309153645e-09 points=2020000000
Step 102: pi=3.1413905176 sigma^2=3.5059444896e-09 points=2040000000
Step 103: pi=3.1413898854 sigma^2=3.4816649781e-09 points=2060000000
Step 104: pi=3.1413935769 sigma^2=3.4555235739e-09 points=2080000000
Step 105: pi=3.1413945314 sigma^2=3.4292454005e-09 points=2100000000
Step 106: pi=3.1413991811 sigma^2=3.4011940269e-09 points=2120000000
Step 107: pi=3.1413974206 sigma^2=3.3743347403e-09 points=2140000000
Step 108: pi=3.1414004907 sigma^2=3.3466366293e-09 points=2160000000
Step 109: pi=3.1413962532 sigma^2=3.3211146426e-09 points=2180000000
Step 110: pi=3.1413969818 sigma^2=3.2956500077e-09 points=2200000000
Step 111: pi=3.1413970757 sigma^2=3.2705160761e-09 points=2220000000
Step 112: pi=3.1413945071 sigma^2=3.2468612623e-09 points=2240000000
Step 113: pi=3.1413941858 sigma^2=3.2236755487e-09 points=2260000000
Step 114: pi=3.1413977737 sigma^2=3.1993105982e-09 points=2280000000
Step 115: pi=3.1414050400 sigma^2=3.1730315529e-09 points=2300000000
Step 116: pi=3.1414101103 sigma^2=3.1461596613e-09 points=2320000000
Step 117: pi=3.1414129026 sigma^2=3.1193688552e-09 points=2340000000
Step 118: pi=3.1414150102 sigma^2=3.0928533353e-09 points=2360000000
Step 119: pi=3.1414165580 sigma^2=3.0666966669e-09 points=2380000000
Step 120: pi=3.1414179733 sigma^2=3.0409356601e-09 points=2400000000
Time = 130.776500

```

Рисунок 8 – Запуск для двох процесорів з N = 120

```

Step 48: pi=3.1413848583 sigma^2=3.2533266483e-09 points=1920000000
Step 49: pi=3.1413827714 sigma^2=3.1856769075e-09 points=1960000000
Step 50: pi=3.1413891380 sigma^2=3.1209693514e-09 points=2000000000
Step 51: pi=3.1413904275 sigma^2=3.0590672395e-09 points=2040000000
Step 52: pi=3.1413870173 sigma^2=2.9991345703e-09 points=2080000000
Step 53: pi=3.1413861679 sigma^2=2.9414698369e-09 points=2120000000
Step 54: pi=3.1413867278 sigma^2=2.8860004365e-09 points=2160000000
Step 55: pi=3.1413789927 sigma^2=2.8333181069e-09 points=2200000000
Step 56: pi=3.1413677232 sigma^2=2.7873552840e-09 points=2240000000
Step 57: pi=3.1413661632 sigma^2=2.7438277682e-09 points=2280000000
Step 58: pi=3.1413750934 sigma^2=2.6970885876e-09 points=2320000000
Step 59: pi=3.1413845695 sigma^2=2.6505870602e-09 points=2360000000
Step 60: pi=3.1413925883 sigma^2=2.6067389904e-09 points=2400000000
Time = 73.419621

```

Рисунок 9 – Запуск для чотирьох процесорів з N = 60

Як можна побачити з рисунків 7, 8 та 9 помилка складає 0.0001 – 0.0002, що є більшим, а ніж 10^{-4} степені. Це пов'язано з рандомністю запусків та самою природою алгоритму. Для отримання гарантованих результатів треба вказувати N в 2 – 3 рази більше, але це, в свою чергу, буде займати в 2-3 рази більше часу виконання.

На таблиці 4 показані швидкості алгоритмів та прискорення з різним N та різною кількістю процесорів

Таблиця 4 – Порівняння прискорень алгоритмів

N	P	T1	Tn	S
240	1	233,024407	233,024407	1
120	2	233,024407	130,7765	1,7818523
60	4	233,024407	73,419621	3,17387101

Як можна побачити з таблиці 4, прискорення при двох процесорах складає 178%, а при 3-х процесорах, при порівнянні з одним, 317%. Це означає, що цю задачу можна гарно розпаралелити на багато процесорів для швидкого виконання.

На рисунку 10 показаний графік залежності прискорення від кількості процесорів. Важливо зауважити, що параметр N також повинен бути налаштований для потрібної кількості процесорів

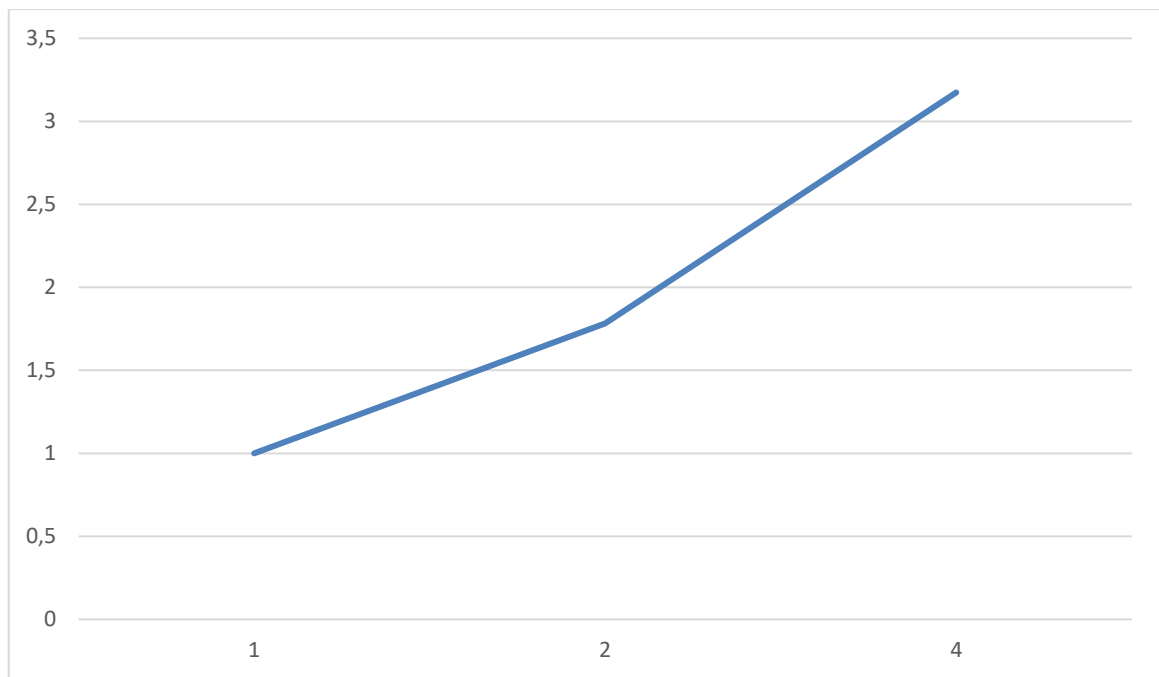


Рисунок 10 – Залежність прискорення від кількості процесорів при налаштованих N

Для виконання наступного завдання 2.3 було взято вихідний код програми 3 та переписано його на паралельну версію через MPI. До цього коду було додано розрахунок для часу виконання алгоритму, а також для дисперсії.

На рисунку 11 показано приклад виконання програми на 10-ти процесорах. Значення N не змінювалось та дорівнює 1 мільярду. Кожен процесор виконує N/P операцій, а потім виконується сума на нульовому процесорі

```

C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 10 .\lb5.exe
2: Processor 2 will do 100000000 points
3: Processor 3 will do 100000000 points
5: Processor 5 will do 100000000 points
1: Processor 1 will do 100000000 points
9: Processor 9 will do 100000000 points
6: Processor 6 will do 100000000 points
8: Processor 8 will do 100000000 points
4: Processor 4 will do 100000000 points
7: Processor 7 will do 100000000 points
0: Processor 0 will do 100000000 points
PI=3.1415624265320097   Error=0.000030227057783           Points = 1000000000
Dispersion = 0.4136030904380554
Time = 8.562581

```

Рисунок 11 – Монте-Карло для розрахунку π через інтеграли

Як можна побачити з рисунку 11, дисперсія складає 0.4136, що правильно при використанні інтегралів. Помилка ж складає 10^{-4} .

Для того, щоб значення гарантовано потрібно взяти N :

$$\varepsilon = \frac{\sigma}{\sqrt{N}}; \quad N = \frac{\sigma^2}{\varepsilon^2}; \quad N = \frac{0.4136}{(10^{-4})^2} = 41,360,000$$

На рисунку 12 показано приклад виконання програми з $N = 41,360,000$ для одного процесору.

```

C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 1 .\lb5.exe 41360000
0: Processor 0 will do 41360000 points
PI=3.1416537251371701   Error=0.000061071547377           Points = 41360000
Dispersion = 0.4136494712700145
Time = 2.057561

```

Рисунок 12 – Значення π з помилкою 10^{-4} для одного процесору

В таблиці 5 показано залежність прискорення від кількості процесорів

Таблиця 5 – Залежність прискорення від кількості процесорів

P	N	T1	Tn	S
1	41 360 000	2,057561	2,057561	1
2	41 360 000	2,057561	1,104798	1,862386608
4	41 360 000	2,057561	0,561822	3,662300515

Як можна побачити з таблиці 5, при збільшені кількості процесорів прискорення збільшується в рази.

На рисунку 13 показано графік залежності кількості процесорів та прискорення

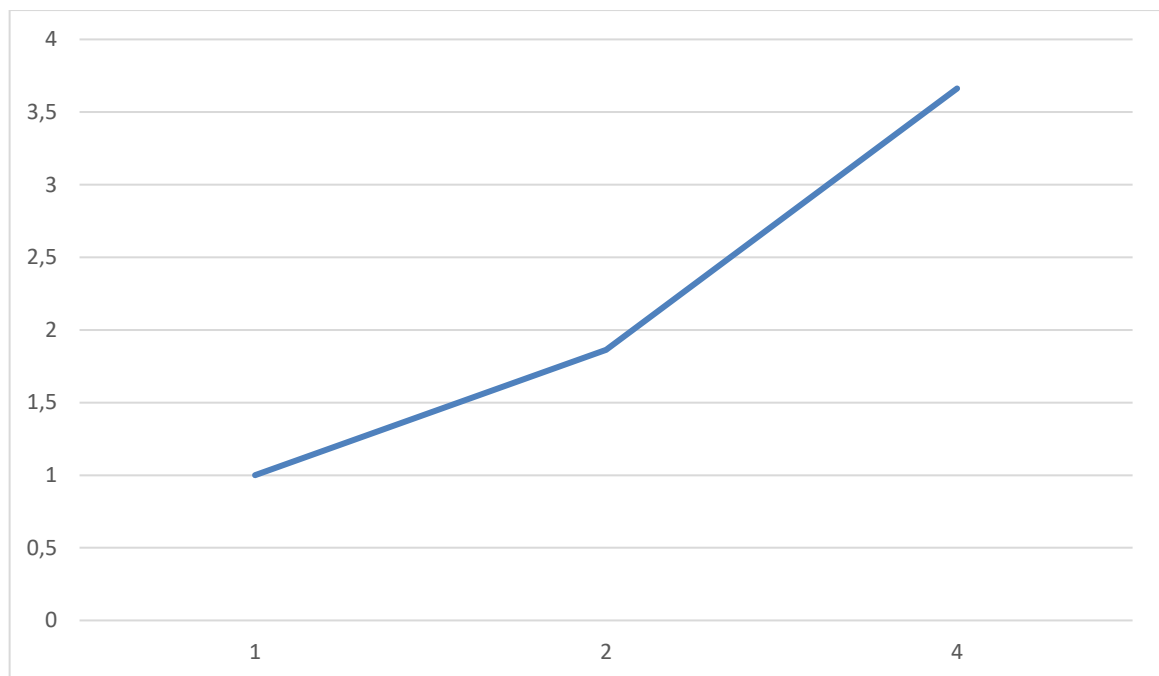


Рисунок 13 – Залежність кількості процесорів та прискорення

Для виконання завдання 2.4 було написано програму, яка розраховує значення кратного інтеграла. В якості стандартного R було взято 1. При такому варіанті точне значення інтеграла дорівнює 0.125, тобто $1/8$.

Програму було запущено на 4-х процесорах з $N = 50,000,000$, це показано на рисунку 14

```
C:\home\university-4\distributed-systems\lb5\x64\Debug>mpiexec -n 4 .\lb5.exe 50000000
Result = 0.1249909156   Error   = 0.0000090844   Expected = 0.1250000000
Dispersion = 0.0170994840
Time = 1.349738
```

Рисунок 14 – Запуск програми на знаходження кратного інтегралу

Як можна побачити, точність є достатньо високою. При збільшені кількості процесорів швидкість алгоритму також буде збільшуватись, оскільки кожен процесор виконує N/P операцій

4 Тест розробленої програми

```
#include "mpi.h"
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <ctime>

int first_original(int argc, char** argv) {
    int rank, numprocs;
    double PI25DT = 3.141592653589793238462643;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    double start;
    if (rank == 0) {
        start = MPI_Wtime();
    }

    int n = 1;
    if (argc > 1) {
        n = atoll(argv[1]);
    }

    double mypi, pi, h, sum, x;

    h = 1.0 / (double)n;
    sum = 0.0;

    for (int i = rank + 1; i <= n; i += numprocs) {
        x = h * ((double)i - 0.5);
        sum += 4.0 / (1.0 + x * x);
    }

    mypi = h * sum;

    MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        double end = MPI_Wtime();
```

```

        printf("%d: pi is approximately %.16f; \t p = %d \t Error is % .16f\t Time:
%fsec \n", rank, pi, numprocs, fabs(pi - PI25DT), end - start);
    }

    MPI_Finalize();
    return 0;
}

int first_sequence(int argc, char** argv) {
    int rank, processors;

    double PI25DT = 3.141592653589793238462643;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &processors);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int n = 1;
    if (argc > 1) {
        n = atoll(argv[1]);
    }

    double start = MPI_Wtime();

    double h = 1.0 / (double)n;
    double sum = 0.0;
    double x;

    for (int i = 1; i <= n; i++) {
        x = h * ((double)i - 0.5);
        sum += 4.0 / (1.0 + x * x);
    }

    double pi = h * sum;
    double end = MPI_Wtime();

    printf("%d: pi is approximately %.16f; \t Error is % .16f\t Time: %fsec \t N = %d\n",
rank, pi, fabs(pi - PI25DT), end - start, n);

    MPI_Finalize();
    return 0;
}

int first_cos(int argc, char** argv) {
    int rank, processors;

    // From wolfram alpha
    double CORRECT = 1.5707963267948966192313216916397514420985846996875529104874722961;
    double PI = 3.141592653589793238462643;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &processors);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int n = 1;
    if (argc > 1) {
        n = atoll(argv[1]);
    }

    double start = MPI_Wtime();

    double a = 0;
    double b = PI;

```

```

double h = (b - a) / (double)n;
double sum = 0.0;

for (int i = 1; i <= n; i++) {
    double x = a + h * ((double)i - 0.5);

    double val = cos(x);
    sum += val * val;
}

double answer = h * sum;
double end = MPI_Wtime();

printf("%d: Answer is approximately %.16f; \t Error is % .16f\t Time: %fsec \t N =
%d\n", rank, answer, fabs(answer - CORRECT), end - start, n);

MPI_Finalize();
return 0;
}

```

```

int second_original(int argc, char* argv[]) {
    const int n_in_proc = 10000000;
    const double R = 0.5;
    const double pi = 3.141592653589793238462643;

    int rank, numprocs;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int N_in_proc = atoi(argv[1]);

    srand(time(NULL) + rank);

    long long in_circle = 0L;
    double sum_xi = 0.0;
    double sum_xi_sq = 0.0;

    double start;
    if (rank == 0) {
        start = MPI_Wtime();
    }

    for (int i = 0; i < N_in_proc; i++) {
        for (int j = 0L; j < n_in_proc; j++) {
            double x = rand() / (double)RAND_MAX - 0.5;
            double y = rand() / (double)RAND_MAX - 0.5;

            if (x * x + y * y < R * R) {
                in_circle++;
            }
        }

        long long total_in_circle;
        MPI_Reduce(&in_circle, &total_in_circle, 1, MPI_LONG_LONG, MPI_SUM, 0,
MPI_COMM_WORLD);

        if (rank == 0) {
            long long total = (long long)n_in_proc * (i + 1) * numprocs;
            double approx = 4.0 * ((double)total_in_circle / total);

            double xi = approx;
            sum_xi += xi;
            sum_xi_sq += xi * xi;
        }
    }
}

```



```

        int current_N = i + 1;
        if (current_N > 1) {
            double variance = (1.0 / (current_N - 1)) * (sum_xi_sq - (1.0 /
current_N) * (sum_xi * sum_xi));
            printf("Step %d: \t pi=%.10f \t sigma^2=%.10e \t", current_N,
approx, variance);
        }

        printf("pi=%.16f; \terror=%.16f, \t points=%lld\n", approx, fabs(pi -
approx), total);
    }
}

if (rank == 0) {
    double end = MPI_Wtime();
    printf("Time = %f", end - start);
}

MPI_Finalize();
return 0;
}

int third(int argc, char* argv[]) {
    const double EXPECTED_PI = 3.141592653589793238462643;

    int rank, processors_count;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &processors_count);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // printf("%d: Hello (p = %d)\n", rank, processors_count);
    srand(time(NULL) + rank);

    long long N = 1000000000;
    if (argc > 1) {
        N = atoll(argv[1]);
    }

    long long perProcess = N / processors_count;
    if (rank == processors_count - 1) {
        perProcess += N % processors_count;
    }
    printf("%d: Processor %d will do %lld points\n", rank, rank, perProcess);

    double start;
    if (rank == 0) {
        start = MPI_Wtime();
    }

    double local_sum = 0.0;
    double local_sum_square = 0.0;
    long long points = 0;

    for (points = 0; points < perProcess; points++) {
        double x = rand() / (double)RAND_MAX;
        double fx = (4.0 / (1.0 + x * x));

        local_sum += fx;
        local_sum_square += fx * fx;
    }

    double sum = 0;
    double sum_square = 0;
    long long points_sum = 0;

```

```

    MPI_Reduce(&local_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&local_sum_square, &sum_square, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
    MPI_Reduce(&points, &points_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        double pi = sum / N;
        double dispersion = (sum_square / N) - (pi * pi);

        printf("PI=%.16lf \tError=%.15lf \tPoints = %lld\n", pi, fabs(EXPECTED_PI -
pi), points_sum);
        printf("Dispersion = %.16lf\n", dispersion);

        double end = MPI_Wtime();
        printf("Time = %f\n", end - start);
    }

    // printf("%d: Bye\n", rank);
    MPI_Finalize();
    return 0;
}

int four(int argc, char* argv[]) {
    double R = 1.0;
    double EXPECTED = 0.125 * pow(R, 4);

    int rank, processors_count;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &processors_count);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    srand(time(NULL) + rank);

    long long N = 1000000000;
    if (argc > 1) {
        N = atoll(argv[1]);
    }

    long long perProcess = N / processors_count;
    if (rank == processors_count - 1) {
        perProcess += N % processors_count;
    }

    double start;
    if (rank == 0) {
        start = MPI_Wtime();
    }

    double local_sum = 0.0;
    double local_sum_sq = 0.0;

    for (long long i = 0; i < perProcess; i++) {
        double x = (double)rand() / RAND_MAX * R;
        double y = (double)rand() / RAND_MAX * R;

        if (x * x + y * y <= R * R) {
            double val = x * y;

            local_sum += val;
            local_sum_sq += val * val;
        }
    }

    double total_sum, total_sum_sq;
    MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

```

```

MPI_Reduce(&local_sum_sq, &total_sum_sq, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    double area_rect = R * R;

    double result = (total_sum / N) * area_rect;
    double variance = (total_sum_sq / N) - pow(total_sum / N, 2);

    printf("Result = %.10f \tError = %.10f \t Expected = %.10f\n", result,
fabs(EXPECTED - result), EXPECTED);
    printf("Dispersion = %.10f\nTime = %f\n", variance, MPI_Wtime() - start);
}

MPI_Finalize();
return 0;
}

int main(int argc, char** argv) {
    // first_original(argc, argv);
    // first_sequence(argc, argv);
    // first_cos(argc, argv);
    // second_original(argc, argv);
    // third(argc, argv);
    four(argc, argv);
}

```

5 Висновки

Я продовжив вивчення функцій обміну бібліотеки MPI, зокрема, функцій колективного обміну. Освоїв деякі прийоми їх використання для розподілу даних та обчислень між паралельними процесорами